# How Chess Arena Works

Start with the one-minute version and go one layer deeper at a time, all the way down to the code. You can stop after any layer and still have a complete picture at that depth.

### How to read this

- **Go top to bottom.** Each layer zooms in one step. Layers 0–4 need no programming knowledge at all. File and function names only show up from Layer 5 on.
- **Stop whenever you like.** Every layer ends with a box called *The short version*. If you can say it back in your own words, you've got that layer.
- **Follow one feature down.** In Layer 1, each feature has *Follow it down* links that jump to the same feature in deeper layers.
- **Dotted words** like WebSocket link to the glossary at the end.
- **Tick "I've read this layer"** to track your progress. It's saved in this browser only.

### Contents

0. The one-minute version
1. What you can do with it
2. The pieces
3. How the pieces talk
4. Each feature, step by step
5. Inside the server
6. Inside the browser app
7. The data
8. How it's deployed
9. Reading the actual code
10. Rough edges
11. Where do I look when… · Glossary · Roadmap

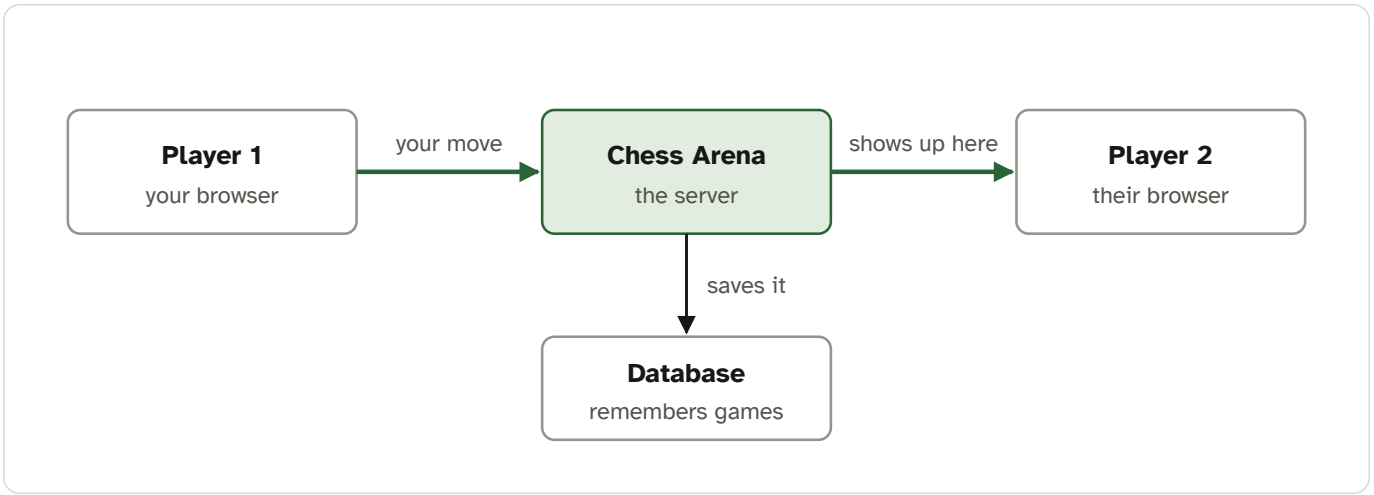
LAYER 0 NO TECH

## The one-minute version

Chess Arena is a website where two people play chess against each other, live. You make an account, press **Play Now**, and the site pairs you with whoever else is waiting. Each of you sees the board from your own side. When one of you moves, the other sees it a moment later.

When a game ends, it's saved. You can open **My Games** later and replay any game move by move.

It lives at `chess.bhargavnkrishna.site`. Behind it are two programs you wrote (a website and a server) and a database that remembers everything.




The whole app in one picture: your move goes to the server, which passes it to your opponent and saves it.

THE SHORT VERSION

Two players, one live board, and every game is saved so you can replay it later.

LAYER 1 NO TECH

# What you can do with it

Every feature, the way a player sees it. Each card's links follow that feature into deeper layers.

### Make an account

Sign up with a username and password, then log in and out. You stay logged in for 24 hours, even if you close the tab.

### Find an opponent

Press **Play Now** and wait. As soon as someone else presses it too, you're both taken to a new game. Whoever waited longest gets matched first. Colors are random.

### Play live

Drag pieces on the board. You can only move your own pieces, only to legal squares. Your opponent's moves appear on their own. A status line says whose turn it is, or that someone is in check.

### Promote a pawn

When your pawn reaches the last row, a picker pops up so you choose queen, rook, bishop or knight. It doesn't just assume a queen.

### Resign, and game over

Press **Resign** to give up. Games also end by checkmate, stalemate and some kinds of draw. Both players see "You won", "You lost" or "Draw".

### Survive hiccups

Refresh the page, lose Wi-Fi for a bit, or even have the server restart. When you're back, the board catches up and the game carries on.

### My Games

A list of every game you've played: when, which color you had, who your opponent was, and whether it's finished.

### Replay a game

Open any game from My Games and step through it with buttons, the arrow keys, or by clicking a move in the list.

SOMETHING YOU'LL NOTICE

If nobody has used the site for a while, the first visit can take about a minute to load. That's the server waking up from sleep, not a bug. Layer 8 explains why.

THE SHORT VERSION

Sign up, get matched, play live with proper promotion and resign, survive refreshes and disconnects, and replay any past game.

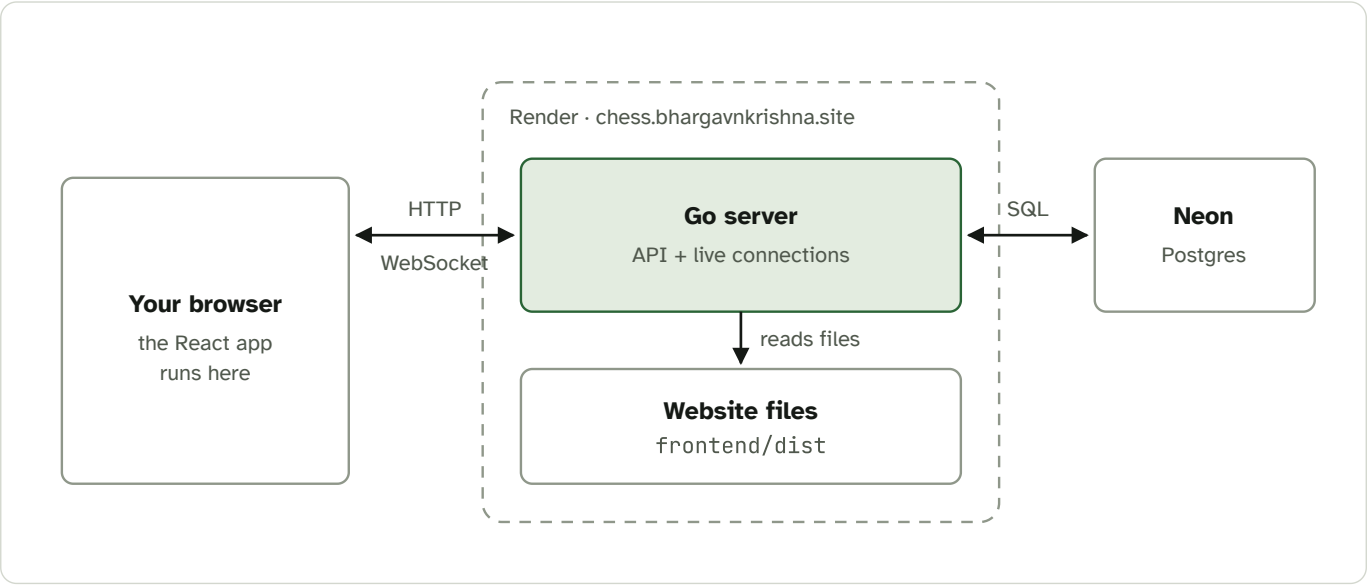
LAYER 2 VERY LIGHT TECH

# The pieces

Four parts make the app work. A restaurant is a handy way to picture them.

| PART | IN THE RESTAURANT | WHAT IT REALLY IS |
|---|---|---|
| **The browser app** (the frontend) | The dining room and the menu. Everything you see and touch. | A React app written in TypeScript. It draws the board, the pages and the buttons inside your browser. |
| **The server** (the backend) | The kitchen. It does the real work and makes the decisions. | A program written in Go. It pairs players, checks every move, passes moves along and saves everything. |
| **The database** | The pantry and the order book. Permanent memory. | A Postgres database hosted by Neon. It stores users, games and every move. |
| **The host** | The building the restaurant is in. | Render. It runs the server on the internet at your domain. |

One detail makes your setup simpler than most: **on Render, the browser app and the server live in the same box.** The server hands out the website's files as well as answering requests. So there's one thing to deploy, one address, and no separate frontend host.




Your browser only ever talks to one place: the Render container at your domain. Only the Go server talks to the database.

## Who's responsible for what

- **The browser app** draws the board, only lets you drag pieces to legal squares, shows the status line, and remembers which color you are.
- **The server** keeps the waiting line, pairs players, checks that every move is legal and that it's your turn, passes moves to your opponent, and saves everything.
- **The database** holds three things: users, games and moves.

KEY IDEA · THE SERVER IS THE REFEREE

Both the browser app and the server know the rules of chess. The browser uses them for convenience, so you can't drag a knight in a straight line. The server's check is the one that counts. If someone tampered with their browser to send a cheating move, the server would refuse it.

KEY IDEA · TWO KINDS OF MEMORY

The server keeps some things in its own short-term memory (fast, but forgotten when it restarts) and some in the database (slower, but permanent). Knowing which is which explains most of the app's behavior. Layer 5 has the full list.

THE SHORT VERSION

A React app in your browser, a Go server that acts as the referee, a Neon database for permanent memory, and Render running the server, which also hands out the website.

# How the pieces talk

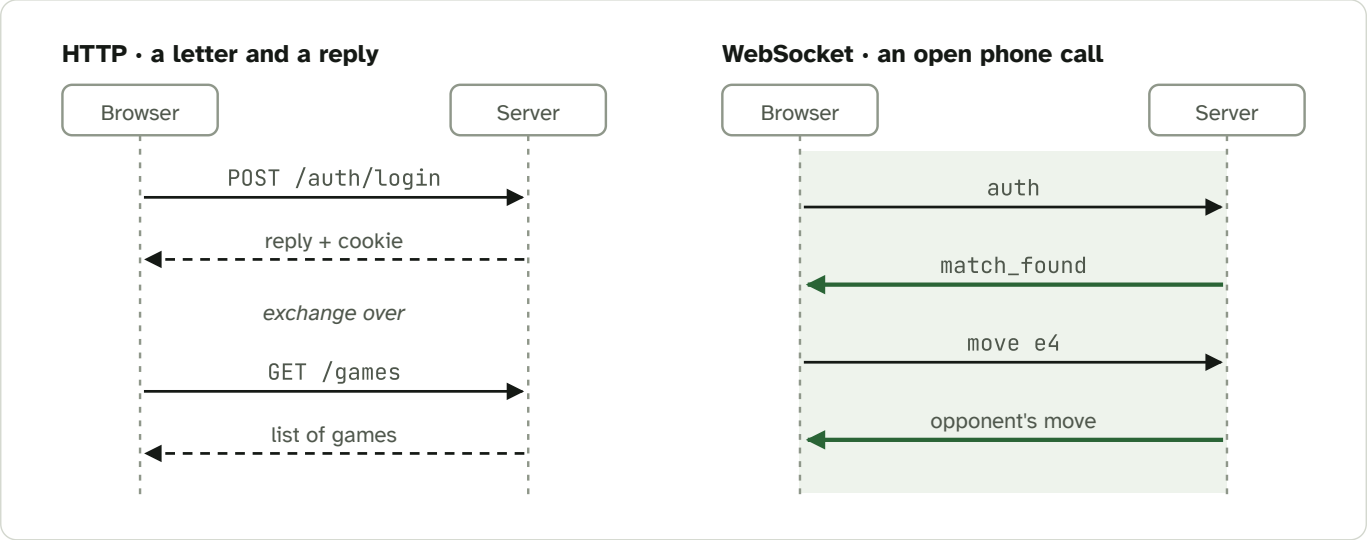
The browser and the server talk in two different ways. You also need to know how the server knows who you are.

## Two ways to talk

**HTTP is like sending a letter.** The browser asks a question, the server sends one reply, and that conversation is over. It's used for things you ask for: sign up, log in, "who am I?", "put me in the waiting line", "show me my games".

**WebSocket is like a phone call that stays open.** Once connected, either side can talk at any moment. It's used for things that happen to you: "your opponent moved", "you've been matched", "game over".

Why both? With HTTP, the server can only *answer*. It can't call you. But when your opponent moves, nobody on your side asked a question. The server needs to speak first, and only the open phone line allows that.




With HTTP the server can only answer. With a WebSocket it can speak first (the green arrows), which is how your opponent's move reaches you.

## Each player has two phone lines

- **The lobby line** is open the whole time you're logged in, on any page. It exists for one message: "you've been matched, here's your game".
- **The game line** opens when you're on a game page. It carries moves, resignations and "game over" for that one game.

Both lines go to the same address on the server ( `/api/ws` ). The first thing the browser says on a new line tells the server which kind it is.

## The wristband: how the server knows it's you

Think of a festival wristband. When you log in, the server gives your browser a signed pass called a JWT, stored in a cookie. It says "this is user 4f2a..., username krishna, valid until tomorrow", plus a signature only the server can make.

- Your browser shows the wristband automatically with every request, and when it opens a phone line.
- The server checks the signature. If it's valid, the server knows who you are without looking anything up.

- The cookie is *HttpOnly*: JavaScript on the page can't read it, so a malicious script can't steal it.
- It expires after 24 hours. Then you log in again.

KEY IDEA · NEVER TRUST WHAT THE BROWSER SAYS ABOUT ITSELF

The server never believes a message that says "I am player X". It only believes the wristband. That's why one player can't send moves pretending to be the other.

## One address for everything

The website, the API ( `/api/…` ) and the phone lines ( `/api/ws` ) all live at `chess.bhargavnkrishna.site`. Because everything is on one domain, the cookie just works everywhere, and there's none of the cross-site setup (CORS) that sites with separate frontend and backend addresses need.

THE SHORT VERSION

HTTP for asking (letter and reply), WebSocket for being told (open phone line). Each player has a lobby line and a game line. A signed cookie, the wristband, proves who you are on every request.

LAYER 4 MEDIUM

# Each feature, step by step

The features from Layer 1 again, now as short stories of what happens behind the scenes. Still no code.

## Signing up and logging in

1. You type a username and password. The browser sends them to the server ( `POST /api/auth/register` ).
2. The server checks the name isn't taken. It scrambles the password with bcrypt and stores only the scrambled version, never the real password.
3. The server creates your wristband (the JWT, valid 24 hours) and puts it in a cookie in its reply.
4. The browser now knows you're logged in and shows your name in the top bar.

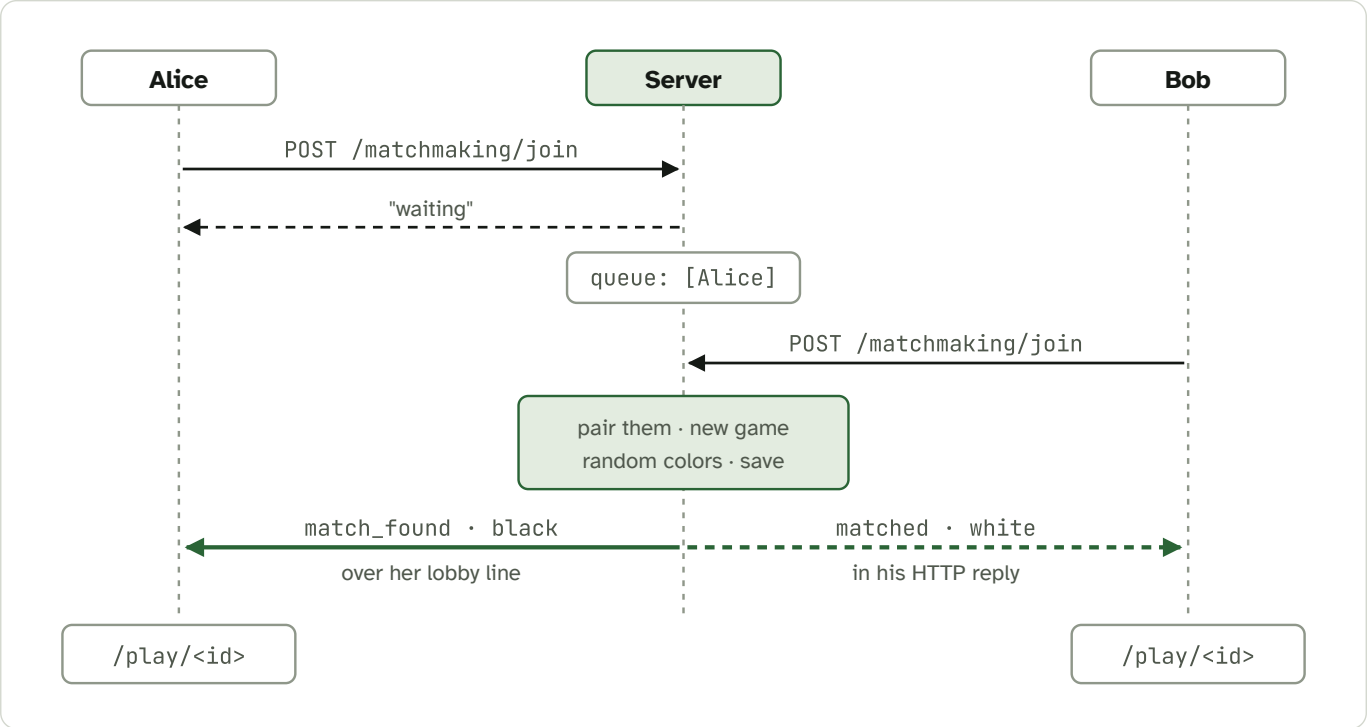
Logging in is the same, except in step 2 the server scrambles the password you typed and checks it matches the stored one.

**Too many attempts?** To slow down password guessing, the server allows 5 quick tries, then one every 2 seconds, per internet address. Too fast and you get "too many requests". (Whether this works properly on Render still needs checking: see Layer 10.)

**Coming back the next day:** when the site loads, the app asks the server "who am I?" ( `GET /api/auth/me` ). Valid wristband: you're in. Expired: you're sent to the login page.

## Finding a match

Say Alice presses Play Now first, then Bob.




Alice and Bob learn about the same match in two different ways, because only Bob is still waiting for a reply.

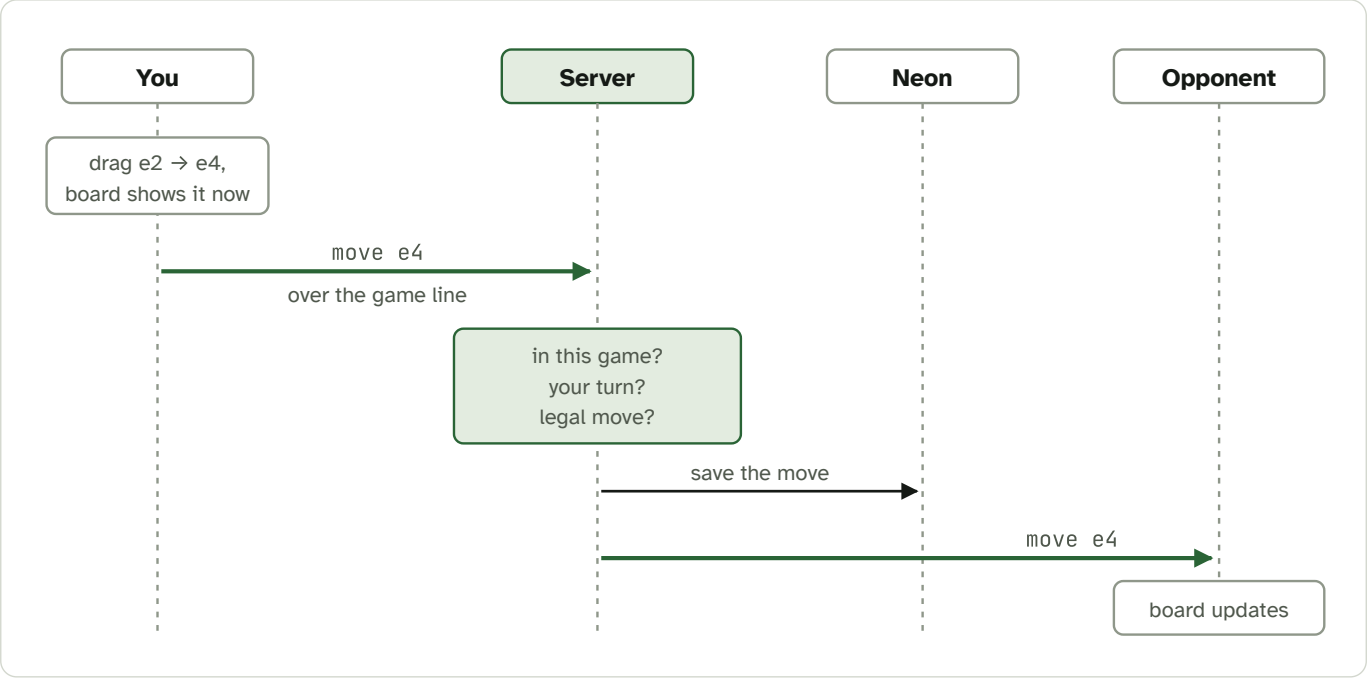
1. Alice asks to join. Nobody is waiting, so she goes into the line and gets the reply "waiting". Her request is now finished.
2. Bob asks to join. Alice is waiting, so the server takes her out of the line and pairs them.

3. The server creates a new game, randomly decides who's white, and saves both players and colors to the database.
4. Alice's request was answered long ago, so the only way to reach her now is her **lobby line**. The server sends her `match_found` with her color.
5. Bob is still waiting for his reply, so he gets "matched" plus his color right in it. (He's also sent a `match_found` on his lobby line. Whichever arrives first moves him along.)
6. Both browsers save their color for this game and open `/play/<gameId>`.

**What if Alice's lobby line happens to be down right then?** The server puts the message aside (a "pending" message) and delivers it the moment she reconnects.

## Making a move

This is the heart of the app.




Your board updates straight away. The server checks the move and saves it before passing it to your opponent.

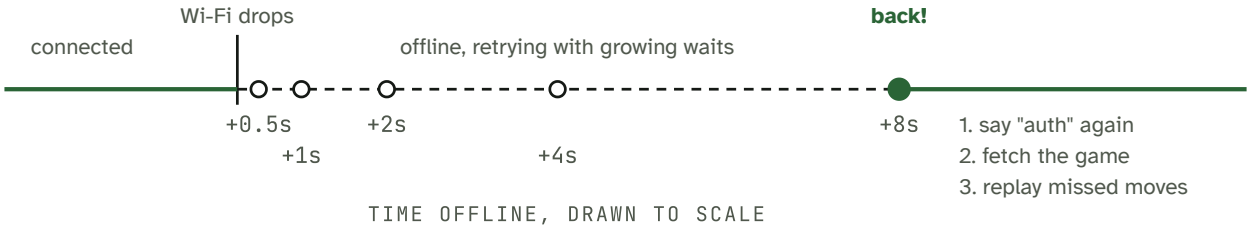
1. You drag a piece. The board only lets you drop it on a legal square, because the browser app knows the rules too.
2. Your board shows the move immediately, without waiting for the server. This is called an "optimistic" update and it's what makes the game feel instant.
3. The browser sends the move over your game line: `{"type":"move", … "move":"e4", "from":"e2", "to":"e4"}`.
4. The server plays referee: are you one of the two players? Is it your turn? Is the move legal in the current position?
5. If it's all fine, the server saves the move to the database and updates its copy of the board.
6. The server forwards the move to your opponent's game line, and their board updates.
7. If that move ended the game (checkmate, for example), the server also sends "game over" to both of you.

If any check in step 4 fails, the server quietly ignores the move. It doesn't tell you, which is one of the rough edges in Layer 10.

## Game over

- **By the board:** after every move, the server's chess library looks at the position. Checkmate, stalemate and certain draws end the game automatically.
- **By resigning:** you press Resign and confirm. The browser sends `{"type":"resign"}`. The server records that you resigned, so your opponent wins.
- Either way the server marks the game *finished* in the database and sends `game_over` with the winner ( `white` , `black` or `draw` ) to both players.
- Each browser compares the winner with its own color and shows "You won", "You lost" or "Draw" on top of the board.

## When your Wi-Fi drops




The browser keeps retrying, waiting twice as long each time (capped at 30 seconds). Once it's back, it catches up from the server.

1. The browser notices the game line closed.
2. It tries to reconnect after 0.5 seconds, then 1, 2, 4, 8... up to 30 seconds between tries. Doubling the wait is called exponential backoff, and it stops thousands of browsers from hammering a struggling server at once.
3. When the line is back, the browser introduces itself again ("I'm here for game X").
4. It then asks for the whole game over HTTP and compares: "the server has 14 moves, I have 12". It plays the 2 missing moves on its board.

Refreshing the page works the same way: your board starts empty and catches up from the server in one go.

## When the server goes to sleep or restarts

On Render's free plan the server shuts down after about 15 minutes with no visitors, and the next visitor wakes it up. The same thing happens on every new deploy.

| SAFE (IN THE DATABASE) | FORGOTTEN (WAS IN MEMORY) |
|---|---|
| Users and passwords | People waiting in the matchmaking line |
| Every game: players, colors, status | Open phone lines (browsers reconnect on their own) |
| Every move ever played | "Match found" messages not yet delivered |
| | The server's quick-access copy of each live board |

**How an unfinished game comes back:** the first time anyone touches that game after a restart, the server reads its moves from the database and replays them on a fresh board, in order. It's like setting up a board again from a written scoresheet. Then play simply continues.

## My Games and Replay

- **My Games** asks the server for your games ( `GET /api/games` ). The server looks up every game where you were one of the players, adds your opponent's name, and sorts newest first.
- **Replay** fetches one game with all its moves ( `GET /api/games/<id>` ). To show move 17, the browser starts from the opening position and plays moves 1 to 17. Buttons and arrow keys just change that number.

Replay needs nothing special from the server. A replay is just the saved list of moves, played back.

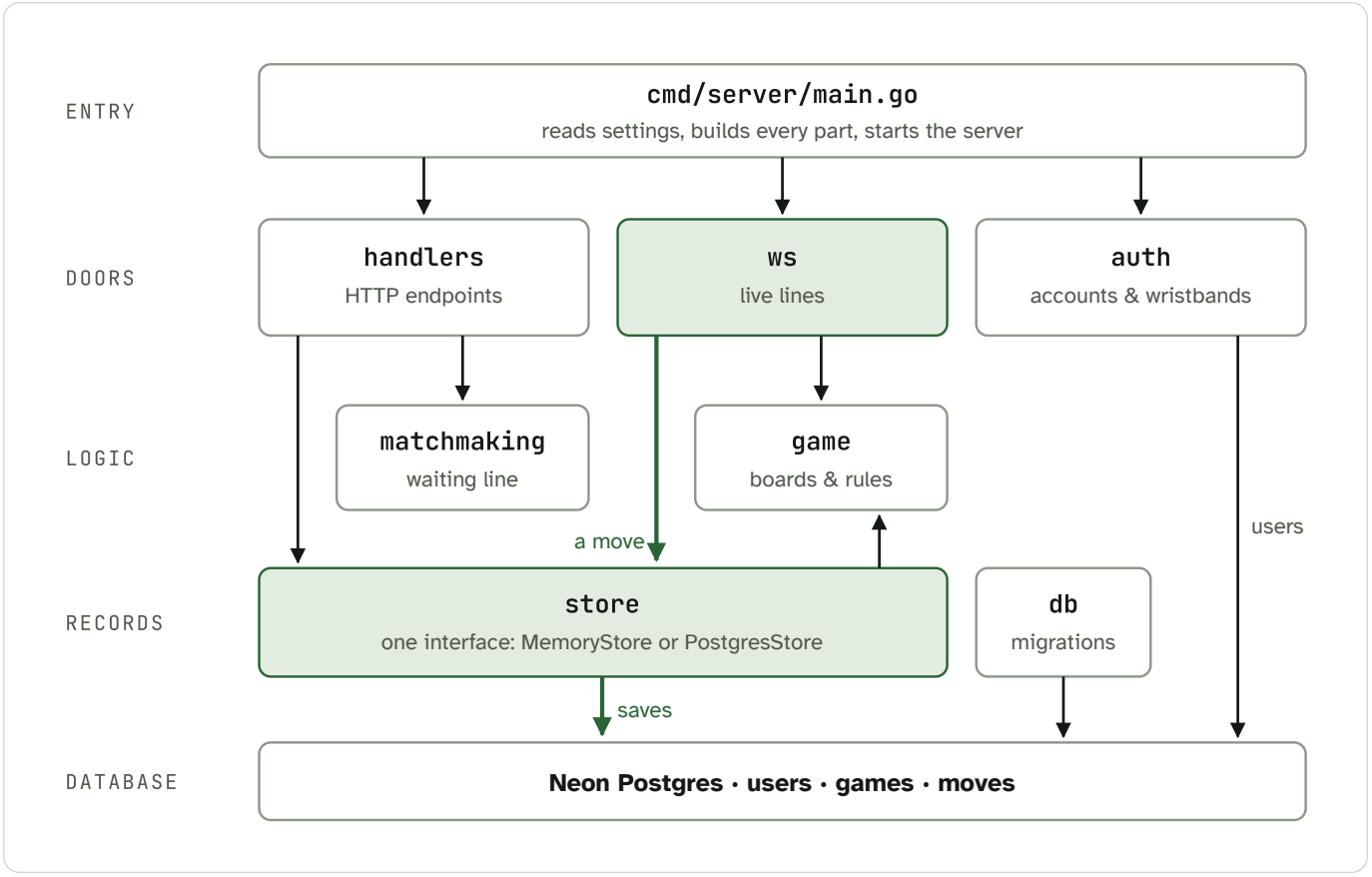
THE SHORT VERSION

Login scrambles your password and hands you a wristband. Matchmaking reaches the first player over the lobby line and the second in the HTTP reply. A move shows on your board instantly, gets checked and saved by the server, then forwarded. Disconnects retry with growing waits and then catch up. Restarts lose only short-term things, and games are rebuilt from saved moves.

LAYER 5 MEDIUM · CODE NAMES START HERE

# Inside the server

The server is one Go program in `backend/`. It's split into folders called packages, and each one has a single job. Think of them as departments in a building.




Arrows mean "uses". Everything is built once in `main.go`. Requests come in through a door and end up in the store. The green path is a move: `ws` gives it to `store`, which checks it with a `game` board and saves it to Neon.

## The departments

| PACKAGE | ITS JOB | MAIN FILES |
|---|---|---|
| `cmd/server` | Reception. Reads settings (environment variables), decides between database mode and memory-only mode, creates every part once, sets up the URL routes, serves the website files, starts listening, and shuts down cleanly. | `main.go` |
| `auth` | Accounts and wristbands. Creates and checks JWTs, sets the cookie, handles register/login/logout/me, stores users (memory or Postgres), hashes passwords with bcrypt, and rate-limits login attempts. | `jwt.go`, `middleware.go`, `handler.go`, `store.go`, `postgres_store.go`, `ratelimit.go` |
| `handlers` | The HTTP endpoints for games and matchmaking: join the queue, get a game, list your games, get moves, make a move over HTTP. | `api.go` |
| `ws` | The phone lines. `hub.go` is the switchboard: who's connected, and messages waiting for someone who isn't. `handler.go` runs each line: check the wristband, open it, read the first message, then handle moves and resignations, with a ping every 45 seconds to keep it alive. | `hub.go`, `handler.go` |
| `matchmaking` | The waiting line. A list of player IDs. Joining either returns a pair or puts you in the line. | `queue.go` |
| `game` | `Session` is one live chessboard that wraps the notnil/chess library: apply a move, whose turn, outcome, resign. `GameManager` is a quick in-memory lookup of who is in which game with which color. | `session.go`, `manager.go` |
| `store` | Records. A single `Store` interface lists what the server needs (create a game, get it, list games, apply a move, whose turn, set players, outcome, resign). Two versions fulfil it: `MemoryStore` (everything forgotten on restart) and `PostgresStore` (live boards in memory, every change written to Neon). | `memory.go`, `postgres.go` |
| `db` | Connects to Postgres and runs migrations: the three `.sql` files, each exactly once, tracked in a `schema_migrations` table. | `db.go`, `migrations/*.sql` |
| `models` | The shapes of data sent as JSON: `Game` and `MoveRecord`. | `game.go` |

## What's in memory and what's in the database

| THING | LIVES IN | SURVIVES A RESTART? |
|---|---|---|
| Users | Neon · `users` | **Yes** |
| Games (players, colors, status, board snapshot) | Neon · `games` | **Yes** |
| Moves | Neon · `moves` | **Yes** |
| Live chessboards ( `game.Session` ) | Memory, inside `PostgresStore` | Rebuilt from moves when needed |
| Who's in which game ( `GameManager` ) | Memory | **No** |
| The waiting line ( `Queue` ) | Memory | **No** |
| Open lines and pending messages ( `Hub` ) | Memory | **No** |
| Rate-limit counters | Memory | **No** |

When `GameManager` has forgotten a game after a restart, the `ws` code falls back to asking the store, which reads the players and colors from Neon. That's why games keep working after the server wakes up.

KEY IDEA · WHY THERE ARE LOCKS EVERYWHERE

Go handles every connection in its own goroutine, a lightweight thread. Two players can touch the same game at the same instant. A mutex ( `sync.Mutex` ) makes them take turns, so the board, queue and switchboard never get half-updated. You'll see `mu.Lock()` in almost every package.

KEY IDEA · WHY THE STORE IS AN INTERFACE

The handlers only know "there is something that can create games, apply moves…". They don't know or care whether it's memory or Postgres. `main.go` picks one at startup based on whether `DATABASE_URL` is set, and nothing else has to change.

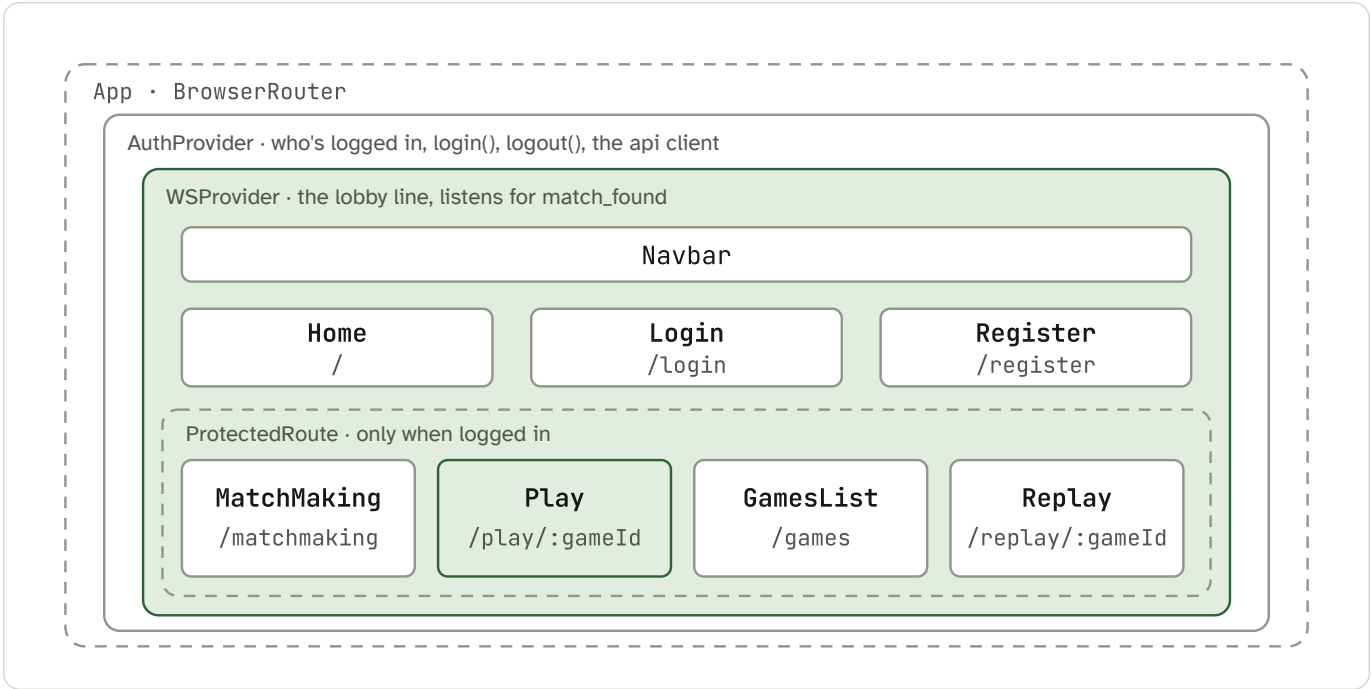
THE SHORT VERSION

`main.go` builds everything. Requests come in through `handlers` (HTTP), `ws` (live lines) and `auth`. `matchmaking` and `game` hold the logic, and `store` saves to Neon. Permanent things are in Neon. The queue, connections and lookups are in memory.

LAYER 6 MEDIUM

# Inside the browser app

The frontend lives in `frontend/src`. It's built with Vite, which turns it into plain HTML, JavaScript and CSS files in `frontend/dist`. Those are the files the Go server hands out.




Boxes inside boxes: every page sits inside the two always-on helpers, so any page can ask "who's logged in?", and a match can pull you into a game from anywhere.

## The pages

| ADDRESS | COMPONENT | WHAT IT DOES | TALKS TO THE SERVER |
|---|---|---|---|
| `/` | `Home` | Welcome screen with a Play Now button. | None |
| `/login`, `/register` | `Login`, `Register` | Forms that call `login()` or `register()` from the auth helper. | `POST /auth/login`, `/auth/register` |
| `/matchmaking` | `MatchMaking` | Joins the queue once. If matched right away, goes to the game. Otherwise shows "Waiting..." and lets the lobby line do the rest. | `POST /matchmaking/join` |
| `/play/:gameId` | `Play` | The live game (details below). | Game line + `GET /games/:id` |
| `/games` | `GamesList` | Table of your games with a Replay link each. | `GET /games` |
| `/replay/:gameId` | `Replay` | Board in view-only mode plus step controls and arrow keys. | `GET /games/:id` |

## The always-on helpers

React has a feature called context for sharing things with every page without passing them down by hand.

- `AuthContext` holds the logged-in `user` and creates one shared `api` client (axios pointed at `/api`, with cookies switched on). When the site loads, it calls `/auth/me` to restore your session. It also provides `login()`, `register()` and `logout()`.
- `WSContext` opens the lobby line as soon as you're logged in. When `match_found` arrives, it saves your color and navigates to the game.

- `ProtectedRoute` wraps the private pages. While the session check is running it shows nothing. If you're not logged in, it sends you to `/login`.

## The game page, piece by piece

- `useChessGame` (a hook) keeps a chess.js game and gives the page the position (FEN), the move list, a status sentence ("White to move", "Checkmate — Black wins"), and functions like `move()`.
- `ChessBoard` wraps chessground, the draggable board from lichess. It's given the position, the legal destination squares for every piece (`buildDests`), and which color you're allowed to move.
- `Play` ties it together:
  - Opens the game line with `ReconnectingWebSocket`. Every time it connects, it sends `auth` with the game ID, then calls `resync()` to catch up.
  - On an incoming `move`, it applies it. On `game_over`, it shows the result overlay.
  - When you move: if it's a pawn reaching the last row, it shows the promotion picker first. Then `sendMove()` applies the move locally and sends it.
  - Resign asks you to confirm, then sends `resign`.
  - Your color comes from `localStorage` (`color_<gameId>`), saved during matchmaking. It decides which way the board faces.
- `GameInfo` and `GameControls` show the status, move list and position, plus the buttons.
- `lib/reconnectingWebSocket.ts` is the retry-with-backoff wrapper from Layer 4. `lib/wsUrl.ts` builds `wss://your-site/api/ws` on HTTPS and `ws://…` on plain HTTP, so it works locally and live with no settings.

KEY IDEA · USEREF VERSUS USESTATE

The chess.js game object is changed in place, so it lives in a `useRef` (a box React doesn't watch). After every move, the hook copies the new FEN, move list and status into `useState`, and changing state is what makes React redraw the page.

THE SHORT VERSION

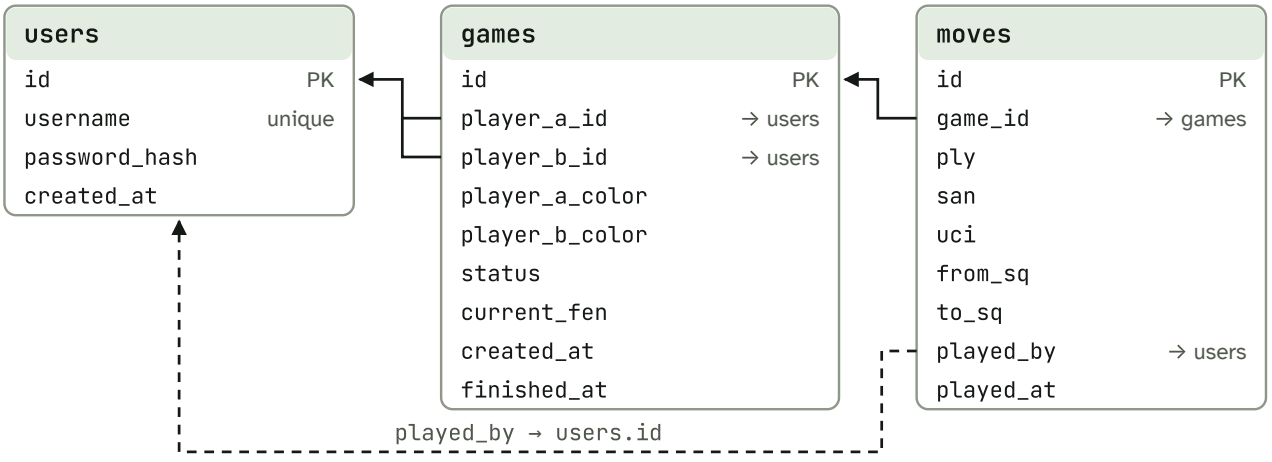
Six pages inside two always-on helpers: Auth (who you are) and WS (the lobby line). The game page combines a chess.js hook, a chessground board and a reconnecting game line that catches up on every connect.

LAYER 7 MEDIUM-DEEP

# The data

What's stored, what's sent, and what every message looks like. Keep this layer open as a reference while reading code.

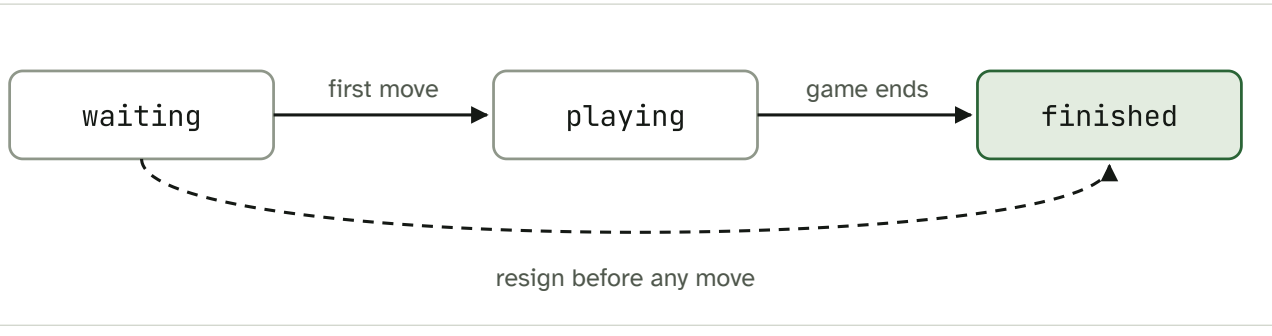
## The three tables




Each game points at its two players, and each move points at its game and the player who made it. The server also keeps a small `schema_migrations` table to remember which `.sql` files have run.

- **Player A** is whoever joined the queue first, **player B** second. Colors are picked randomly, so A isn't always white.
- `current_fen` is a snapshot of the board after the latest move, updated on every move.
- `ply` numbers each half-move from `0`. Ply `0` is white's first move, ply 1 black's reply, and so on.
- The tables are created by `backend/internal/db/migrations/001_create_users.sql`, `002_create_games.sql` and `003_create_moves.sql`.

### A game's status




A freshly matched game says `waiting` until someone moves, so it shows up that way in My Games.

## Chess notation the app uses

| NAME | WHAT IT IS | EXAMPLE |
|---|---|---|
| FEN | The whole board in one line: pieces, whose turn, castling rights, move counters. | `rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3 0 1` |
| SAN | How people write moves. This is what browsers send to the server. | `e4`, `Nf3`, `exd5`, `O-O`, `e8=Q` |
| UCI | From-square plus to-square, with a promotion letter if needed. Stored alongside SAN. | `e2e4`, `e7e8q` |

## HTTP endpoints

All of these live under `/api` on your domain. "Login" means the request must carry a valid wristband cookie.

| METHOD | PATH | LOGIN? | PURPOSE |
|---|---|---|---|
| POST | `/auth/register` | — | Create an account, set the cookie. Rate-limited. |
| POST | `/auth/login` | — | Check password, set the cookie. Rate-limited. |
| POST | `/auth/logout` | — | Clear the cookie. |
| GET | `/auth/me` | cookie | "Who am I?" Returns your ID and username. |
| POST | `/matchmaking/join` | yes | Join the queue. Returns "waiting", or "matched" with a game ID and your color. |
| GET | `/games` | yes | Your games, newest first, with opponent names. |
| GET | `/games/:id` | — | One game: players, colors, status, position, every move. |
| GET | `/games/:id/moves` | yes | Just the move list. |
| POST | `/games/:id/move` | yes | Make a move over HTTP. The app doesn't use it (it uses the game line). |
| POST | `/games` | — | Creates an empty game. A leftover: nothing in the app calls it. |
| GET | `/healthz` (not under /api) | — | Returns `ok` if the server is up and can reach the database. |
| WS | `/ws` | cookie | Opens a lobby line or a game line. |

`POST /api/matchmaking/join · the two possible replies`

```
{"Status": "waiting"}

{"Status": "matched", "gameId": "9b1c…", "playerColor": "white"}
```

`GET /api/games/:id · shortened`

```
{
  "id": "9b1c…",
  "playerA": "4f2a…", "playerB": "c07e…",
  "playerAUsername": "alice", "playerBUsername": "bob",
  "playerAColor": "black", "playerBColor": "white",
  "status": "playing",
  "initial_fen": "rnbqkbnr/pppp1ppp/8/4p3/4P3/8/PPPP1PPP/RNBQKBNR w KQkq e6 0 2",
  "moves": [
    {"ply": 0, "san": "e4", "uci": "e2e4", "from": "e2", "to": "e4", "by": "c07e…", "timestamp": "…"},
    {"ply": 1, "san": "e5", "uci": "e7e5", "from": "e7", "to": "e5", "by": "4f2a…", "timestamp": "…"}
  ],
  "createdAt": "…"
}
```

Watch out: the field called `initial_fen` actually holds the *current* position. It's a naming slip in `models/game.go`.

## WebSocket messages

Every message has the same envelope: a `type`, usually a `gameId`, and a `payload`.

| DIRECTION | TYPE | EXAMPLE |
|---|---|---|
| browser → server | `auth` (lobby) | `{"type":"auth","payload":{}}` |
| browser → server | `auth` (game) | `{"type":"auth","payload":{"gameId":"9b1c…"}}` |
| browser → server | `move` | `{"type":"move","gameId":"9b1c…","payload":{"move":"e8=N","from":"e7","to":"e8","promotion":"n"}}` |
| browser → server | `resign` | `{"type":"resign","gameId":"9b1c…"}` |
| server → browser | `match_found` | `{"type":"match_found","gameId":"9b1c…","payload":{"playerColor":"black"}}` |
| server → browser | `move` | The same message the mover sent, forwarded as is. |
| server → browser | `game_over` | `{"type":"game_over","gameId":"9b1c…","payload":{"outcome":"white"}}` (or `black`, `draw`) |

The server checks the SAN in `move` (here `e8=N`). The opponent's browser applies `from`, `to` and `promotion`. There are also invisible "ping" and "pong" frames every 45 seconds to keep the line alive. Your code never sees those.

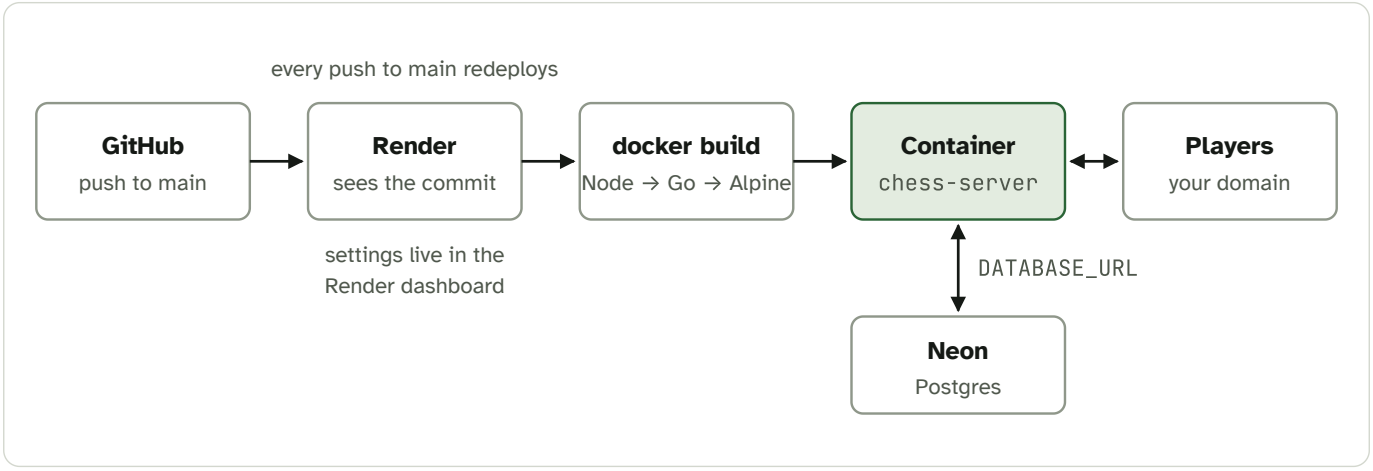
THE SHORT VERSION

Three tables: users, games (pointing at two users) and moves (pointing at a game). Moves travel as SAN. Boards are stored as FEN. About ten HTTP endpoints, plus four message types going up the line and three coming down.

LAYER 8 MEDIUM-DEEP

# How it's deployed

How your code gets from GitHub to `chess.bhargavnkrishna.site`, what settings it needs, and why some files in the repo aren't used.




Auto-deploy is on for `main`, so merging to `main` updates the live site. Pushing to any other branch doesn't.

## The Dockerfile's three stages

A Dockerfile is a recipe for building a container image. Yours has three stages, and only the result of the last one is kept:

1. `node:20-alpine` installs the frontend's packages and runs `npm run build`. Out come plain files in `frontend/dist`.
2. `golang:1.25-alpine` compiles the backend into a single program, `chess-server`. `CGO_ENABLED=0` makes it fully self-contained.
3. `alpine:latest` is a tiny final image. It copies in just the program and the `dist` folder, then runs `/app/chess-server`.

When it starts, the Go server sends `/api/…` to the API, `/healthz` to the health check, and everything else to the website files. Unknown paths like `/play/abc` get `index.html`, so React can show the right page after a refresh.

## Settings (environment variables)

These are set in Render's dashboard under **Environment**, never committed to the repo.

| VARIABLE | WHAT IT DOES | IF IT'S WRONG |
|---|---|---|
| `DATABASE_URL` | Connection string for Neon. | Missing: the server runs in memory-only mode and forgets everything on every sleep. |
| `JWT_SECRET` | The secret used to sign wristbands. At least 32 characters. | Missing or short: the server refuses to start. Leaked: anyone could forge a wristband. |
| `COOKIE_SECURE` | `true` makes the browser send the cookie only over HTTPS. | Should be `true` live. Locally it's `false` because there's no HTTPS. |
| `ALLOWED_ORIGIN` | The site's own address, `https://chess.bhargavnkrishna.site`. Used for CORS and to check where WebSocket connections come from. | Mismatch: phone lines are refused, so matching and live moves stop working. |
| `PORT` | Which port to listen on. Render sets it for you. | Defaults to 8080. |
| `STATIC_DIR` | Where the website files are. | Defaults to `frontend/dist`, which matches the Dockerfile. |

## What the free plans mean

- **Render free:** after about 15 minutes with no visitors, the server is stopped. The next visitor waits while it starts again, "50 seconds or more" according to your dashboard banner. Layer 4 covers what's lost.
- **Neon free:** the database also pauses when idle and wakes on the first query, which adds a little delay to that first request.
- `/healthz` pings the database too, so it tells you whether both are awake and connected.

## Local development versus live

| | ON YOUR LAPTOP (`./dev.sh`) | LIVE ON RENDER |
|---|---|---|
| Database | Postgres in Docker (`docker-compose.yml`) | Neon |
| Backend | `go run` on port 8080 | Compiled `chess-server` in the container |
| Frontend | Vite dev server on port 5173, reloads as you edit | Built files served by the Go server |
| How `/api` reaches Go | Vite forwards `/api` (and WebSockets) to port 8080 (`vite.config.ts`) | Same program, no forwarding needed |
| Cookie | Not Secure (plain HTTP) | Secure (HTTPS) |

## Files the live site doesn't use

**How they got there.** On 30 August you first set the app up to run on your own machine: `prod.sh` started everything, Caddy routed traffic, and a Cloudflare tunnel exposed it to the internet. That commit (`8f5283e`, "deployment") is also the one Render tried to deploy first, and it failed because there was no Dockerfile yet. About an hour and a half later, `a63f7d6` added the Dockerfile and taught the Go server to hand out the website files itself. That's what's live, and it made the self-hosting files unnecessary.

Cleanup is on hold, so here's what each leftover is. None of them affect the live site.

| FILE OR CODE | WHAT IT WAS FOR | USED NOW? |
|---|---|---|
| `Caddyfile` | Caddy's routing rules: `/api/*` and `/healthz` to the Go server, everything else to the website files, listening only on the local machine. | **No. Go does this itself now.** |
| `prod.sh` | Self-hosting launcher: start Postgres, build both halves, run Go and Caddy. | **No** |
| `deploy/cloudflared.yml.example` | Config for a Cloudflare tunnel to expose your machine at your domain. | **No** |
| `frontend/bun.lockb` | Lockfile from an early experiment with the Bun package manager. | **No.** `package-lock.json` **(npm) is the real one.** |
| `frontend/README.md` | Vite's template readme, not about your app. | **No** |
| `frontend/src/assets/*`, `public/icons.svg` | Template images ( `hero.png` , `react.svg` , `vite.svg` ). | **No. Nothing imports them.** |
| `frontend/.example.env` | Empty file. | **No** |
| `frontend/src/lib/pgnDisplay.ts` | Helpers to format a full PGN with headers. | **No. Nothing imports it.** |
| `react-chessboard` in `package.json` | An alternative board library. | **No. The app uses chessground.** |
| `POST /games` ( `api.CreateGame` ) | Create an empty game. Needs no login. | **No** |
| `GameManager.playerToGame`, `Queue.ClearQueue` | A lookup and a helper that are written but never read or called. | **No** |

Two small related things: the browser tab title is still Vite's default, "frontend" ( `frontend/index.html` ). And a few code comments still talk about the Cloudflare tunnel, which matters for one of the rough edges in Layer 10.

THE SHORT VERSION

Push to `main` , Render builds the three-stage Dockerfile and runs one container that serves both the API and the website, talking to Neon through `DATABASE_URL` . Free plans mean cold starts. Caddy, `prod.sh` and the tunnel config are leftovers from a first attempt at self-hosting.

LAYER 9 ■■■■■■■■■ DEEP

# Reading the actual code

Guided tours of the nine most important pieces of code, with real excerpts. // … marks lines left out to keep things short. Read these with the file open next to you.

---

## 1 · Starting up: picking database or memory

backend/cmd/server/main.go

```
if dbURL := os.Getenv("DATABASE_URL"); dbURL ≠ "" {
    pool, err := db.Connect(dbURL)
    // … stop if it fails …
    if err := db.Migrate(pool); err ≠ nil { /* … */ }
    st = store.NewPostgresStore(pool)
    userStore = auth.NewPostgresUserStore(pool)
} else {
    logger.Println("DATABASE_URL not set — using in-memory stores")
    st = store.NewMemoryStore()
    userStore = auth.NewUserStore()
}
// …
root.Handle("/api/", http.StripPrefix("/api", mux))
root.Handle("/", spaHandler(getenv("STATIC_DIR", "frontend/dist")))
```

- `st` has the type `store.Store` (the interface), so either version fits.
- Migrations run on every startup but skip files already applied.
- `StripPrefix` removes `/api`, so the inner routes are written as `/games`, `/auth/login` and so on.

## 2 · Checking the wristband

backend/internal/auth/middleware.go

```
func Middleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        claims, err := TokenFromRequest(r)       // read cookie, verify signature + expiry
        if err ≠ nil {
            http.Error(w, "unauthorized", http.StatusUnauthorized)
            return
        }
        ctx := context.WithValue(r.Context(), UserIDKey, claims.UserID)
        ctx = context.WithValue(ctx, UsernameKey, claims.Username)
        next.ServeHTTP(w, r.WithContext(ctx))    // hand over, with your ID attached
    })
}
```

- This is the middleware pattern: a handler that wraps another handler. In `main.go` you'll see `auth.Middleware(http.HandlerFunc(api.ListGames))`.
- Later code calls `auth.UserIDFromContext(r.Context())` to get your ID. It never comes from the request body.

## 3 · Matchmaking

backend/internal/matchmaking/queue.go

```
func (q *Queue) JoinQueue(playerID string) *MatchResult {
    q.mu.Lock()
    defer q.mu.Unlock()
    if slices.Contains(q.waiting, playerID) {
        return nil                              // already waiting
    }
    if len(q.waiting) > 0 {
        playerA := q.waiting[0]                 // first in line
        q.waiting = q.waiting[1:]
        return &MatchResult{PlayerA: playerA, PlayerB: playerID}
    }
    q.waiting = append(q.waiting, playerID)
    return nil
}
```

backend/internal/handlers/api.go · JoinMatchMaking, shortened

```
res := a.Queue.JoinQueue(playerID)
if res == nil {
    writeJSON(w, http.StatusOK, struct{ Status string }{Status: "waiting"})
    return
}
g, err := a.Store.CreateGame()
// …
a.Manager.AddGame(g.ID, res.PlayerA, res.PlayerB)   // random colors picked here
gm := a.Manager.GetGame(g.ID)
_ = a.Store.UpdateGamePlayers(g.ID, res.PlayerA, res.PlayerB, gm.PlayerAColor, gm.PlayerBColor)
// … build msgA / msgB with each player's color …
_ = a.Hub.SendToLobby(res.PlayerA, msgA)            // Alice: lobby line
_ = a.Hub.SendToLobby(res.PlayerB, msgB)
writeJSON(w, http.StatusOK, map[string]interface{}{  // Bob: the HTTP reply
    "Status": "matched", "gameId": g.ID, "playerColor": gm.PlayerBColor,
})
```

- The lock makes "check the line and take someone out" one step, so two people joining at the same instant can't both grab Alice.
- The `_ =` means errors are ignored here. If a lobby line is down, the Hub stores the message as pending instead (tour 6).

## 4 · Opening a phone line

backend/internal/ws/handler.go · ServeWS, shortened

```
claims, err := auth.TokenFromRequest(r)   // wristband checked BEFORE upgrading
if err ≠ nil { http.Error(w, "unauthorized", http.StatusUnauthorized); return }
playerID := claims.UserID

conn, err := upgrader.Upgrade(w, r, nil)  // HTTP becomes a WebSocket here
conn.SetReadLimit(1024)

var initMsg Message
if err := conn.ReadJSON(&initMsg); err ≠ nil || initMsg.Type ≠ "auth" {
    _ = conn.Close(); return
}
// … read payload.GameID, set up ping/pong deadlines, start pingLoop …
if payload.GameID == "" {
    serveLobby(h, conn, playerID)
    return
}
serveGame(h, conn, st, manager, playerID, payload.GameID)
```

- `upgrader.CheckOrigin` only accepts connections whose `Origin` header equals `ALLOWED_ORIGIN`, so other websites can't open a line using your cookie.
- A game ID in the first message decides lobby versus game.

- `pingLoop` pings every 45 seconds. If no pong arrives within 70 seconds, the read times out and the line closes.

## 5 · A move arriving at the server

`backend/internal/ws/handler.go · inside serveGame`

```
if msg.Type == "move" {
    var movePayload MovePayload
    if err := json.Unmarshal(msg.Payload, &movePayload); err ≠ nil { continue }

    color, ok := playerColor(manager, st, msg.GameID, playerID)
    if !ok { continue }                                  // not in this game
    if !isPlayersTurn(st.GetTurn(msg.GameID), color) { continue }
    if _, err := st.ApplyMove(msg.GameID, movePayload.Move, playerID); err ≠ nil {
        continue                                         // illegal move
    }
    opponentID, ok := opponentOf(manager, st, msg.GameID, playerID)
    if !ok { continue }
    h.SendToGame(opponentID, msg.GameID, msg)            // forward as is

    if outcome := st.Outcome(msg.GameID); outcome ≠ "" {
        broadcastGameOver(h, msg.GameID, outcome, playerID, opponentID)
    }
}
```

- The three referee checks from Layer 4 are the three `continue` lines. Each one silently drops the move (Layer 10).
- `st.ApplyMove` does the legality check (through `game.Session` and notnil/chess) and saves to Neon in one call.
- `playerColor` and `opponentOf` try the in-memory `GameManager` first, then fall back to the store after a restart.

## 6 · The switchboard

`backend/internal/ws/hub.go`

```
func (h *Hub) register(key string, conn *websocket.Conn) {
    h.mu.Lock()
    defer h.mu.Unlock()
    if old, ok := h.clients[key]; ok {
        _ = old.Close()                    // same key again: kick the old line
    }
    h.clients[key] = conn
    if msg, ok := h.pending[key]; ok {     // anything waiting for you?
        _ = conn.WriteJSON(msg)
        delete(h.pending, key)
    }
}

func (h *Hub) sendTo(key string, msg Message) error {
    h.mu.Lock()
    conn, ok := h.clients[key]
    h.mu.Unlock()
    if ok {
        if err := conn.WriteJSON(msg); err ≠ nil {   // write happens outside the lock
            // … forget the conn, keep msg as pending …
        }
        return nil
    }
    h.mu.Lock()
    h.pending[key] = msg                   // not connected: keep it for later
    h.mu.Unlock()
    return errors.New("no-conn, stored pending")
}
```

- Keys are `playerID` for the lobby and `playerID:gameID` for a game line.
- Each key holds only one pending message. A second one overwrites the first.
- Writing outside the lock is the root of a rough edge (Layer 10).

## 7 · Rebuilding a game after a restart

`backend/internal/store/postgres.go · getOrLoadSession, shortened`

```
s.mu.RLock()
sess, ok := s.sessions[gameID]
s.mu.RUnlock()
if ok {
    return sess, nil                        // already in memory: fast path
}
// … check the game exists in Postgres …
rows, err := s.db.Query(context.Background(), `SELECT san FROM moves WHERE game_id=$1 ORDER BY ply`, gameID)
rebuilt := game.NewSession()
for rows.Next() {
    var san string
    rows.Scan(&san)
    rebuilt.ApplyMove(san, "")              // replay the scoresheet
}
s.mu.Lock()
if existing, ok := s.sessions[gameID]; ok {
    rebuilt = existing                      // someone else rebuilt it first: use theirs
} else {
    s.sessions[gameID] = rebuilt
}
s.mu.Unlock()
```

- Every `PostgresStore` method that needs a board starts with this, so rebuilding happens on first use.
- The second check under the write lock handles two requests for the same game arriving at once after a restart.

## 8 · The reconnecting socket

`frontend/src/lib/reconnectingWebSocket.ts`

```
this.socket.onopen = () => {
  const wasReconnect = this.hasConnectedBefore;
  this.hasConnectedBefore = true;
  this.attempt = 0;                        // reset the backoff
  this.onOpen(wasReconnect);
};
this.socket.onclose = () => {
  if (this.intentionallyClosed) { this.onClose(); return; }
  this.scheduleReconnect();
};

private scheduleReconnect() {
  const delay = Math.min(this.initialBackoffMs * Math.pow(2, this.attempt), this.maxBackoffMs);
  this.attempt++;                          // 500, 1000, 2000, 4000 … capped at 30000
  this.reconnectTimer = window.setTimeout(() => this.connect(), delay);
}
```

- `close()` sets `intentionallyClosed`, so leaving a page doesn't trigger reconnect attempts.
- Messages sent while the line is down are dropped with a warning, not queued.

## 9 · Catching up on the game page

`frontend/src/components/Play.tsx`

```
const resync = async () => {
  const res = await api.get(`/games/${gameId}`);
  const serverMoves = res.data.moves ?? [];
  const localCount = game.history().length;
  for (let i = localCount; i < serverMoves.length; i++) {
    move(serverMoves[i].from, serverMoves[i].to);   // apply only the missing ones
  }
  if (res.data.status === "finished") setGameOver("finished");
};

const ws = new ReconnectingWebSocket(wsUrl("/api/ws"), {
  onOpen: () => {
    ws.send(JSON.stringify({ type: "auth", payload: { gameId } }));
    resync();                                       // on first connect AND every reconnect
  },
  // … onMessage: "move" → move(…), "game_over" → overlay …
});
```

- Starting the loop at `localCount` makes it safe to run as often as you like.
- Notice `move()` gets no promotion piece here, so it falls back to a queen (Layer 10).

THE SHORT VERSION

Startup picks a store. Middleware attaches your ID. The queue pairs under a lock. The WebSocket handler checks the wristband, then routes lobby or game. A move passes three checks, gets saved, then forwarded. The Hub keeps lines and pending messages. Games rebuild from the moves table. The browser retries with backoff and resyncs every time it connects.

LAYER 10 ■■■■■■■■■■ DEEP

# Rough edges

Things that aren't quite right yet, worst first. Every real project has a list like this. Each item says what happens, why, and how it could be fixed.

---

## The login limiter may trust a header anyone can fake SECURITY · NEEDS CHECKING

**What happens** Possibly, someone could make unlimited password guesses by sending a different made-up `CF-Connecting-IP` header with each request.

**Why** `clientIP()` in `auth/ratelimit.go` trusts that header first. It was written for the Cloudflare tunnel, where Cloudflare always overwrites it with the real address. On Render it depends on what sits in front of the app. If Cloudflare is in front (your own Cloudflare proxy, or Render's edge if it uses Cloudflare), the header is still trustworthy. If not, it comes straight from whoever sent the request. And without the header, every request may appear to come from Render's own proxy, so all users would share one limit.

**Check** Temporarily log `clientIP(r)`, then send a login request with a fake `CF-Connecting-IP: 1.2.3.4` header. If the log shows `1.2.3.4`, the hole is real.

**Fix** Read the client address only from a header your host is guaranteed to set and overwrite.

---

## Two writers on one connection can break a line BUG · RARE

**What happens** If two messages are written to the same WebSocket at the same instant (for example one player resigns exactly as the other moves), gorilla/websocket panics with "concurrent write to websocket connection". Go's HTTP server catches that panic, so the whole server keeps running, but that player's line drops and a message can be lost or garbled. The browser then reconnects and resyncs.

**Why** The gorilla/websocket library allows only one writer per connection at a time. `Hub.sendTo` writes outside its lock, and each player's goroutine can write to the other's connection.

**Fix** Give each connection one writer goroutine fed by a channel, or a small mutex per connection that every write takes.

---

## Rejected moves fail silently BUG

**What happens** If the server refuses a move, your board still shows it and your opponent never receives it. The two boards disagree until you refresh.

**Why** Each check in `serveGame` just does `continue`. The browser already applied the move optimistically, and `resync()` only adds missing moves, it never removes extra ones.

**Fix** Send a `move_rejected` message back and have the browser undo the move or reload the game.

---

## Under-promotions turn into queens after a reconnect BUG

**What happens** Promote to a knight, then refresh or reconnect: your board now shows a queen, while the server correctly has a knight.

**Why** `resync()` calls `move(from, to)` without the promotion piece, and `move()` defaults to `"q"`.

**Fix** Use the fifth character of the stored `uci` (e.g. `e7e8n`), or apply the stored `san` directly.

### The move that's forwarded isn't the move that was checked BUG

| | |
|---|---|
| What happens | A tampered client could send SAN `e4` (which passes the check) with `from` / `to` of a different move. The opponent's board shows the wrong move. |
| Why | The server validates `payload.move` but forwards the original message, and the opponent's browser applies `from` / `to`. |
| Fix | Forward what the server actually applied: the `MoveRecord` returned by `ApplyMove` (from, to, uci). |

---

### Two tabs keep kicking each other off BUG

| | |
|---|---|
| What happens | Open the site in two tabs of the same browser: their lobby lines take turns disconnecting each other about every half second. The same happens with one game open in two tabs. |
| Why | `Hub.register` closes any existing line with the same key. The kicked tab reconnects after 0.5 seconds and kicks the other one back. |
| Fix | Allow several lines per key (a list instead of one), or tell the old tab "replaced" with a special close code so it stops reconnecting. |

---

### Your color is only remembered in one browser BUG

| | |
|---|---|
| What happens | Open a game from another browser or device (or after clearing site data) and the board assumes you're white. If you're actually black, it faces the wrong way, lets you drag white's pieces, and the server rejects those moves. |
| Why | `Play.tsx` reads the color from `localStorage` and defaults to white. |
| Fix | Work it out from `GET /games/:id`, which already includes both players' IDs and colors, the way `Replay.tsx` does. |

---

### The waiting line never forgets you LIMITATION

| | |
|---|---|
| What happens | Press Play Now, then leave. You stay in line. When the next person joins, you're paired with them. If you're still on the site, you get pulled into a game. If you've left, they sit in a game alone. |
| Why | `Queue.LeaveQueue` exists but nothing calls it. |
| Fix | Call it when the matchmaking page closes (a small leave endpoint) or when the lobby line disconnects. |

---

### Your screen can say "draw" while the server keeps playing LIMITATION

| | |
|---|---|
| What happens | On a threefold repetition, the status line says "Draw by repetition", but no game-over message arrives and moves are still accepted. |
| Why | chess.js in the browser reports threefold repetition. The server's notnil/chess only ends a game by itself on automatic draws (such as fivefold repetition or insufficient material). Threefold is a draw a player has to claim. |
| Fix | Add a "claim draw" message the server can accept, or have the server treat threefold as final. |

## Saving a move is two separate writes LIMITATION

| | |
|---|---|
| What happens | If the second write fails, the game's board snapshot and its move list disagree, and the in-memory board is already a move ahead. |
| Why | `PostgresStore.ApplyMove` runs `UPDATE games` and then `INSERT INTO moves` without a transaction. |
| Fix | Wrap both in one transaction. On failure, drop the in-memory session so it's rebuilt from the database. |

## Smaller things TIDY-UP

- **Only one pending message per player.** A second message overwrites the first. Mostly harmless, because `resync()` recovers moves.
- **Messages can name a different game than their line.** `serveGame` uses the `gameId` inside each message rather than the one the line was opened for. You still have to be a player in that game, so it's untidy rather than dangerous.
- **Undo, New Game and Flip Board on the game page.** Undo and New Game change only your own board, putting it out of sync. Flip does nothing.
- **Any database error reads as "not found".** `GetGame` returns `ErrorNotFound` for every error, including a Neon hiccup while it wakes up.
- `GET /games/:id` **needs no login.** Anyone with a game's ID can read it. IDs are random UUIDs, so they're hard to guess. Just make it a deliberate choice.
- `POST /games/:id/move` isn't used, doesn't notify the opponent, and returns 404 after a restart because it only checks the in-memory `GameManager`.
- `initial_fen` holds the current position. Rename it to `current_fen`.
- **No automated tests yet.** `matchmaking/queue.go` and `game/session.go` are the easiest place to start.

THE SHORT VERSION

Check the rate limiter's IP source first (security), then fix the concurrent-write problem. After that, the sync bugs: silent rejections, promotions on resync, forwarding the checked move, tab fights and remembered color. The rest is housekeeping.

REFERENCE

# Where do I look when...

| SOMETHING'S UP WITH… | LOOK HERE |
| --- | --- |
| Can't log in, or keep getting logged out | `auth/handler.go`, `auth/jwt.go` (24-hour expiry, `COOKIE_SECURE`), `context/AuthContext.tsx` |
| "Too many requests" | `auth/ratelimit.go` |
| Stuck on "Waiting for other players" | `matchmaking/queue.go`, `handlers/api.go` (JoinMatchMaking), `context/WSContext.tsx`, `ws/hub.go` (pending) |
| Board faces the wrong way | `localStorage` key `color_<gameId>`, `components/Play.tsx` |
| A move doesn't reach the opponent | `Play.tsx` (sendMove), `ws/handler.go` (serveGame checks), `ws/hub.go` (sendTo) |
| Boards out of sync after a reconnect | `Play.tsx` (resync), `lib/reconnectingWebSocket.ts` |
| Game didn't come back after a restart | `store/postgres.go` (getOrLoadSession), Neon dashboard |
| Live moves don't work in production | `ALLOWED_ORIGIN` on Render, `ws/handler.go` (CheckOrigin), `lib/wsUrl.ts` |
| Site is slow to open | Render free-plan cold start (Layer 8) |
| Server won't start on Render | Render logs. Check `JWT_SECRET` length and `DATABASE_URL`. |
| Changing the database | Add `backend/internal/db/migrations/004_….sql`. It runs once on the next start. |
| Adding a new WebSocket message | `ws/handler.go` (serveGame) and `Play.tsx` (onMessage) |

REFERENCE

# Glossary

| | |
|---|---|
| **axios** | A JavaScript library for making HTTP requests. The app uses one shared instance called `api`. |
| **Exponential backoff** | Retrying with a wait that doubles each time (0.5s, 1s, 2s…), usually with a cap. |
| **bcrypt** | A deliberately slow way to scramble passwords so stolen hashes are hard to crack. You can check a password against it, but you can't unscramble it. |
| **Caddy** | A web server often used to route traffic. Your repo has a `Caddyfile` from the self-hosting attempt. It isn't used on Render. |
| **chessground** | The board UI library from lichess. It handles drawing and dragging, but not the rules. |
| **chess.js** | A JavaScript chess rules library used in the browser for legal moves and game status. |
| **Container** | A packaged, isolated environment holding a program and everything it needs. Render runs yours. |
| **Context (React)** | A way to share values (like the logged-in user) with every component below a provider. |
| **Cookie** | A small value the browser stores and sends back automatically. *HttpOnly* means scripts can't read it. *Secure* means it's only sent over HTTPS. *SameSite=Lax* limits sending it from other sites. |
| **CORS** | Browser rules about one website calling another website's API. Mostly irrelevant here, because everything is on one domain. |
| **Docker / Dockerfile** | A tool and a recipe file for building container images. |
| **FEN** | Forsyth–Edwards Notation: a one-line description of a chess position. |
| **Frontend / backend** | The frontend is the part that runs in your browser. The backend is the part that runs on the server. |
| **Goroutine** | Go's lightweight thread. The server runs one per connection, so many games happen at once. |
| **Hook (React)** | A function starting with `use` that gives a component state or behavior, like `useChessGame`. |
| **HTTP** | The request-and-reply protocol of the web. One request, one response. |
| **Interface (Go)** | A list of methods. Any type that has those methods fits, which is how `MemoryStore` and `PostgresStore` are interchangeable. |
| **JWT** | JSON Web Token: a small signed piece of data ("user X, valid until Y"). The server can verify it without a database lookup. |
| **Migration** | A numbered SQL file that changes the database structure. Each one runs exactly once. |
| **Mutex** | A lock that lets only one goroutine at a time touch some shared data. |
| **Neon** | A hosted Postgres service. Your database lives there. |
| **notnil/chess** | The Go chess library the server uses as referee. |
| **Postgres** | PostgreSQL, a popular SQL database. |
| **React** | A JavaScript library for building user interfaces from components. |
| **Render** | The hosting service that builds and runs your container at your domain. |
| **SAN** | Standard Algebraic Notation, how people write moves: `e4`, `Nf3`, `O-O`. |
| **SPA** | Single-page app: one HTML page where JavaScript swaps the content as you navigate, instead of loading new pages. |

| | |
|---|---|
| **UCI notation** | Moves written as from-square plus to-square: `e2e4`, `e7e8q`. |
| **Vite** | The frontend build tool and dev server. |
| **WebSocket** | A connection that stays open so both sides can send messages at any time. |

REFERENCE

# Using this with the roadmap

The repo also has `docs/ROADMAP.md`, a hands-on study plan with exercises. This guide explains, and the roadmap has you try things. Here's how they line up:

| ROADMAP STAGE | READ IN THIS GUIDE |
|---|---|
| 0 · Big picture | Layers 0–3 |
| 1 · Backend entry point | Layer 5, tour 1 in Layer 9 |
| 2 · Authentication | Layer 3 (wristband), Layer 4 (login), tour 2 |
| 3 · Data layer | Layer 5 (memory vs database), Layer 7, tour 7 |
| 4 · Chess engine wrapper | Layer 5 (`game`), Layer 7 (notation) |
| 5 · Matchmaking | Layer 4 (finding a match), tour 3 |
| 6 · WebSockets on the server | Layer 4 (a move), tours 4–6 |
| 7 · The game page | Layer 6, tours 8–9 |
| 8 · History & replay | Layer 4 (replay), Layer 6 (pages) |
| 9 · Deployment | Layer 8 |
| 10 · Capstone | Layer 10: pick a rough edge and fix it |

---

Written from the code on branch `claude/sleepy-johnson-8hetzm` (latest app code: commit `a63f7d6`). If the code changes, this guide can be regenerated from `docs/guide/how-chess-arena-works.html`.